BVS1 – Betriebssysteme

Lecture 2 – From Source to Executables: Towards Processes · Winter Semester 2026/2027

Stavros Kousidis

TH Köln · ICCT

From Source to Executables: Towards Processes

We Already Know the End

Last time we observed:

```
Hello BVS1
```

on an RV64 target with **no guest operating system**.

Today:

> *How did our source code become something that this target could execute?*

And this time we will unfold the build ourselves.

CC BY-SA 4.0

Prediction — What Does QEMU Receive?

Which of these do we give to QEMU to start our RV64 target?

1. `main.c`
2. `start.S`
3. an object file such as `main.o`
4. the linked ELF
5. the Makefile

Commit to one answer before we inspect anything.

CC BY-SA 4.0

Step 1 — Compile the C Source

Create the manual-build directory and compile **without linking**:

```
mkdir -p build/manual
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -msmall-data-limit=0 \
  -c main.c -o build/manual/main.o
```

The other options are course-provided target/build settings.

`-c` means: compile/assemble, but do not link.

CC BY-SA 4.0

Predict Before We Inspect

We have compiled `main.c`, but we have **not linked**.

Before running `file`, predict:

1. **Has the target architecture already been decided?**
2. **Which architecture should `file` report?**
3. **Is `main.o` already the final executable?**

Then test your prediction:

```
file build/manual/main.o
```

CC BY-SA 4.0

What Did `file` Tell Us?

Typical output contains:

```
ELF 64-bit ... relocatable ... RISC-V ...
               |                  |
          not final yet       target ISA
```

ELF = Executable and Linkable Format

So:

- the target architecture is already decided;
- `main.o` is not the final linked program;
- the source was C, but the object file is already target-specific: RISC-V.

CC BY-SA 4.0

Step 2 — Assemble the Startup Code

Translate the startup assembly:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -c start.S -o build/manual/start.o
```

A **symbol** is a name for a program entity or location.

Inspect:

```
riscv64-unknown-elf-nm build/manual/start.o
riscv64-unknown-elf-nm build/manual/main.o
```

Look only for symbols an object **provides** or **still needs**.

CC BY-SA 4.0

Why Can’t We Run the Object Files Yet?

We can already see:

```
start.o defines _start
start.o needs   main
start.o needs   _stack_top

main.o  defines main
```

So one question remains:

> *Who provides **`_stack_top`**, and who decides where all of this lives in target memory?*

CC BY-SA 4.0

Back to the Stored-Program Idea

Lecture 1

```
instructions + data
        ↓
      memory
        ↕
       CPU
```

Now

```
instructions
constant data
writable data
stack  ← runtime memory for
function calls
```

Where should these pieces live in our RV64 memory?

CC BY-SA 4.0

Our RV64 Program in Memory

One memory — different roles for different bytes.

stack = runtime memory used during function calls

`sp` = **stack pointer**

```
RV64 memory from 0x80000000


+-----------------------------------------------------------+
| .text    _start, main, ...      | instructions          |
| .rodata  "Hello BVS1\n"         | constant data         |
| .data                           | initialized data      |
| .bss                            | zero-initialized data |
| C stack  (top: _stack_top)      | runtime data          |
+-----------------------------------------------------------+
```

The Linker Script Makes the Layout Concrete

`linker.ld` is **course-provided**; QEMU provides the machine, not the script.

```
ENTRY(_start)

MEMORY {
    RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 128M
}

. = ALIGN(16);
. += 16K;
_stack_top = .;
```

`.` = current output address. After reserving 16 KiB, that address is named `_stack_top`.

CC BY-SA 4.0

Step 3 — Link the Program

```
riscv64-unknown-elf-gcc -nostdlib -nostartfiles -no-pie \
  -T linker.ld build/manual/start.o build/manual/main.o \
  -o build/manual/hello-bvs1.elf
```

`-T linker.ld` selects our course-provided memory layout.

```
start.o + main.o + linker.ld -> linker -> hello-bvs1.elf
```

The linker **combines objects, resolves symbols, and establishes the final memory layout**.

CC BY-SA 4.0

Inspect the Final ELF

```
file build/manual/hello-bvs1.elf
riscv64-unknown-elf-nm build/manual/hello-bvs1.elf
```

Compare with `start.o`.

Questions:

- Is `main` still unresolved?
- Does `_stack_top` now have an address?
- Which architecture is the executable for?

CC BY-SA 4.0

Where Does Execution Begin?

The linker script told us:

```
ENTRY(_start)
```

And `nm` (“name list”) shows that `_start` now has an address.

So when the RV64 machine starts our program:

> *execution begins at* ***`_start`***

Where did `_start` come from?

Our First RISC-V Assembly

From `start.S`:

```
_start:
    la   sp, _stack_top
    call main
1:
    j    1b
```

Read it as a story:

- set `sp` to `_stack_top`;
- call `main`;
- if `main` returns, stay here forever.

CC BY-SA 4.0

Step 4 — Run It Without Make

```
qemu-system-riscv64 -machine virt -m 128M \
  -smp 1 -nographic -bios none \
  -kernel build/manual/hello-bvs1.elf
```

Expected:

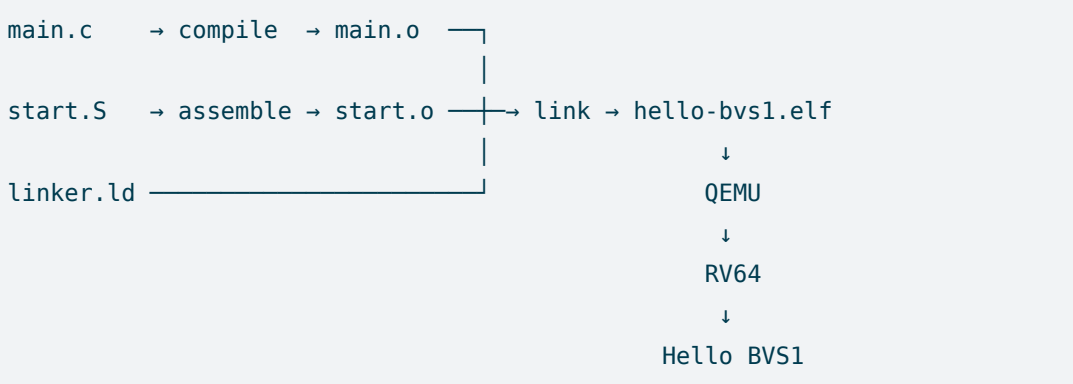
```
Hello BVS1
```

Exit QEMU with **Ctrl-a**, release, then **x**.

We have reproduced the experiment without `make`.

CC BY-SA 4.0

Reconstruct the Complete Path



Which arrows happen on the development host? Which on the target?

Change, Rebuild, Observe

Our First Source-Code Change

Find:

```
void main(void)
{
    uart_puts("Hello BVS1\n");
    for (;;) {}
}
```

Change **only the string**, for example:

```
uart_puts("Hello from <your name>\n");
```

Do not rebuild yet.

CC BY-SA 4.0

Prediction — What Must Change?

We changed only `main.c`.

Which artefacts must change before the new text can appear?

- `main.o`
- `start.o`
- `linker.ld`
- `hello-bvs1.elf`
- QEMU itself

Explain your prediction before rebuilding.

CC BY-SA 4.0

Rebuild Only What Is Necessary

Recompile `main.c`:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -msmall-data-limit=0 \
  -c main.c \
  -o build/manual/main.o
```

Then link again.

Do **not** rebuild `start.o`.

Run the new ELF and compare prediction with observation.

Who Does What?

Make — automates the workflow; it does not compile or link by itself.

- decides whether the target is out of date;
- runs the recipe.

GCC driver

- compiles `main.c`, assembles `start.S`, and invokes the linker.

QEMU

- provides the emulated RV64 machine and loads the ELF.

CC BY-SA 4.0

Toward a Process

One Execution Owns the CPU

Our current path is simple:

```
_start
  ↓
main
  ↓
print
  ↓
for (;;) {}
```

The RV64 CPU keeps executing this one flow forever.

Who can decide to run something else?

CC BY-SA 4.0

The Requirement Changes

Suppose the software we are building should become an operating system.

Then it must be able to:

```
run some execution
       ↓
regain control
       ↓
continue that execution later
```

The execution must therefore become something the OS can **manage**.

Give the Managed Execution a Name

For our first operating-system model:

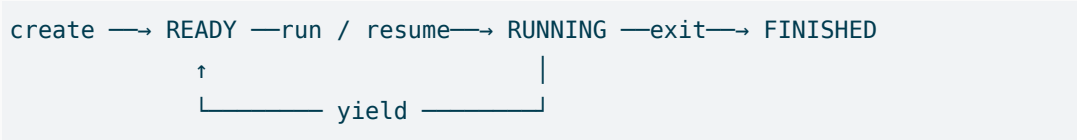
> *A **process** is an execution that the operating system can manage.*

For **Checkpoint 1 — I have a process.**, "manage" means only:

```
create → run → yield → resume → exit
```

Nothing more yet.

Our First Process Lifecycle



A yielded process may run again.

A finished process must not.

CC BY-SA 4.0

Yield Is Not Exit

yield

execution is **not finished**

the OS regains control

the process must be able to continue later

exit

execution is **finished**

the OS regains control

the process must never continue

Same control transfer. Different meaning.

Resume Is Not Restart

Imagine a process executes:

```
uart_puts("A\n");
process_yield();
uart_puts("B\n");
process_exit();
```

`process_yield()` and `process_exit()` are operations we still have to implement.

After `yield`, the OS has control.

When the process runs again:

*Should it print **A** or **B** next?*

The Problem We Have Not Solved

We now want:

```
process runs
     ↓
yield
     ↓
OS regains control
     ↓
resume
     ↓
continue exactly where it stopped
```

What must survive the pause so that this is possible?

CC BY-SA 4.0

Before You Leave

Today we followed one straight path:

```
source → objects → linked ELF → memory → _start → main
```

From there:

```
executing program → OS-managed execution → process
```

A process must be able to:

```
create → run → yield → resume → exit
```

Next:

What machine state makes resume possible?

CC BY-SA 4.0

References

Selected references:

- RISC-V International, *RISC-V ABIs Specification*
- GCC, *RISC-V Options*
- GNU Binutils, *GNU linker (ld)* and *nm* documentation
- QEMU, *RISC-V System Emulator* and `virt` machine documentation
- Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*

CC BY-SA 4.0

Take-Home Concepts

If you remember five things from today:

- Compilation and assembly produce **target-specific object files**.
- The linker **resolves symbols and fixes the memory layout** to create the executable.
- Execution begins at **`_start`**, which establishes the initial machine state before entering C.
- QEMU loads the linked ELF into the **RV64 target**, where its instructions execute.
- An executing program becomes a useful **process abstraction** only when the OS can stop it, regain control, and later **resume rather than restart** it.
  The next question is therefore: what execution state must survive a pause?

CC BY-SA 4.0